



UNIVERSIDADE FEDERAL DA PARAÍBA
CAMPUS IV - RIO TINTO
SISTEMAS DE INFORMAÇÃO

Equipe: Rhauany Aragão, Jamilly Santos, Maria Rafaela, Mariano Santos.

Disciplina: Análise e Projeto de Sistemas

Professor: Dorgival Pereira da Silva Netto

Tema: Sistema de gestão de granja

Agro Plus

Documento de Modelagem de Classes – AgroPlus

1. Arquitetura Utilizada: MVC (Model-View-Controller)

O sistema AgroPlus será desenvolvido seguindo o padrão arquitetural MVC, que separa a aplicação em três camadas principais:

- **Model (Modelo):** Responsável pelos dados e regras de negócio. Contém as classes que representam as entidades do sistema (Animal, Estoque, Produção, etc.) e a lógica de persistência/validação.
- **View (Visão):** Responsável pela interface com o usuário. Apresenta os dados e captura as entradas do usuário.
- **Controller (Controlador):** Intermedia as requisições entre View e Model, processando eventos, validando permissões e direcionando ações.

2. Diagrama de Classes (Descrição Textual)

Abaixo estão descritas todas as classes organizadas por camada, com seus atributos, métodos e relacionamentos.

2.1. Camada Model

Classe	Atributos	Métodos
Usuario	- id: int - nome: string - email: string - senha: string - tipo: string [Administrador, Funcionario, Veterinario] - dataCadastro: Date	+ autenticar(email, senha): boolean + getPermissoes(): list + alterarSenha(novaSenha): void

Animal	- id: int - identificacaoUnica: string - especie: string - idade: int (meses) - pesoAtual: float - situacaoSaude: string [Saudavel, Doente, EmTratamento, Morto] - dataNascimento: Date - lote: string - ativo: boolean	+ cadastrar(dados): void + atualizarPeso(novoPeso): void + atualizarSaude(novaSituacao): void + getHistoricoCompleto(): HistoricoAnimal + excluir(): void (somente admin)
RegistroPeso	- id: int - animalId: int - peso: float - dataPesagem: Date	+ registrar(animalId, peso): void + listarPorAnimal(animalId): list
VacinaMedicamento	- id: int - nome: string - tipo: string [Vacina, Medicamento] - lote: string - dataValidade: Date - quantidadeEstoque: int	+ aplicar(animalId, dose, dataAplicacao): void + atualizarEstoque(quantidade): void
AplicacaoProduto	- id: int - animalId: int - produtoId: int - dose: string - dataAplicacao: Date	+ registrarAplicacao(dados): void

	- veterinarioResponsavel: string - observacoes: string	
VisitaVeterinaria	- id: int - dataVisita: Date - veterinarioId: int - objetivo: string - observacoes: string - animaisAtendidos: list(int)	+ registrarVisita(dados): void + listarVisitasPorPeriodo(data Ini, dataFim): list
AnimalDoente	- id: int - animalId: int - dataDiagnostico: Date - sintomas: string - diagnostico: string - tratamento: string - dataRecuperacao: Date (opcional)	+ registrarDoenca(dados): void + encerrarCaso(dataRecupera cao): void
ProducaoDiaria	- id: int - data: Date (obrigatória) - quantidadeOvos: int (ou quantidade de carne/outro) - lote: string - observacoes: string	+ registrarProducao(dados): void + getProducaoPorPeriodo(dat aIni, dataFim): list + getRelatorioProducao(): Relatorio

Estoque	- id: int - produtoId: int - nomeProduto: string - quantidadeAtual: float - unidadeMedida: string - estoqueMinimo: float - estoqueMaximo: float - localArmazenamento: string	+ registrarEntrada(quantidade, notaFiscal): void + registrarSaida(quantidade, motivo): void + verificarEstoqueBaixo(): boolean + gerarAlerta(): string
Alimento	- id: int - nome: string - tipo: string [Ração, Suplemento, Mineral] - fornecedor: string - validade: Date - precoUnitario: float	+ consumir(animalId, quantidade): void + atualizarPreco(novoPreco): void
Funcionario	- id: int - nomeCompleto: string - cpf: string - funcao: string - telefone: string - dataAdmissao: Date - usuarioid: int (vínculo com autenticação)	+ cadastrar(dados): void + alterarFuncao(novaFuncao): void + demitir(): void

Relatorio	- id: int - tipo: string [Producao, Estoque, Saude, Desempenho] - dataGeracao: Date - parametros: map - conteudo: blob ou texto	+ gerar(tipo, parametros): file + exportarPDF(): void + exportarExcel(): void
HistoricoAnimal	(Classe agregadora, não persistente) - animal: Animal - listaPesos: list(RegistroPeso) - listaAplicacoes: list(AplicacaoProduto) - listaDoencas: list(AnimalDoente) - listaProducoesAssociadas: list(ProducaoDiaria)	+ carregarHistorico(animalId): HistoricoAnimal + exibirCompleto(): string

2.2. Camada View (Principais interfaces)

Classe View	Funcionalidades	Métodos principais
LoginView	Tela de autenticação	+ exibirFormulario() + capturarCredenciais(): (email, senha)
DashboardView	Painel principal com resumos e gráficos	+ exibirResumoProducao() + exibirAlertasEstoque() + exibirAnimaisDoentes()

AnimalView	CRUD de animais, histórico, peso, vacinas	+ listarAnimais() + formCadastroAnimal() + exibirHistorico(animalId) + registrarPeso()
EstoqueView	Controle de entradas/saídas, alertas	+ exibirEstoqueAtual() + formEntradaSaida() + exibirAlertas()
ProducaoView	Registro e consulta de produção diária	+ formRegistroProducao() + exibirGraficoProducao()
RelatorioView	Geração e download de relatórios	+ exibirOpcoesRelatorio() + visualizarRelatorio()
SaudeView	Registro de vacinas, medicamentos, visitas, doenças	+ formVacina() + formVisitaVeterinaria() + listarAnimaisDoentes()
FuncionarioView	Cadastro e gerenciamento de funcionários (admin)	+ listarFuncionarios() + formCadastroFuncionario()

Classe Controller	Responsabilidades	Métodos principais
AuthController	Gerenciar login, logout, sessão, permissões	+ login(email, senha): boolean + logout(): void

		+ verificarPermissao(usuario, acao): boolean
AnimalController	Intermediar operações sobre animais	+ cadastrarAnimal(dados): void + atualizarPeso(animalId, peso): void + obterHistorico(animalId): HistoricoAnimal + excluirAnimal(animalId): void (se admin)
EstoqueController	Controlar estoque e alimentos	+ registrarEntrada(produto, quantidade): void + registrarSaida(produto, quantidade, motivo): void + verificarEstoqueMinimo(): list
ProducaoController	Gerenciar produção diária	+ registrarProducao(dados): void + obterProducaoPeriodo(inicio, fim): list + gerarRelatorioProducao(): Relatorio

SaudeController	Vacinas, medicamentos, visitas, doenças	+ aplicarVacina(animalId, vacinalId, dose): void + registrarVisita(dadosVisita): void + registrarDoenca(animalId, diagnostico, tratamento): void
RelatorioController	Gerar e exportar relatórios	+ gerarRelatorio(tipo, parametros): Relatorio + exportarPDF(relatorioid): file + exportarExcel(relatorioid): file
FuncionarioController	Gerenciar funcionários (admin)	+ cadastrarFuncionario(dados): void + alterarFuncao(funcionarioid, novaFuncao): void + demitirFuncionario(funcionarioid): void
DashboardController	Fornecer dados para o painel principal	+ obterResumoProducao(): map + obterAlertasEstoque(): list

```
+ obterAnimaisDoentes():  
list
```

2.3. Camada Controller

3. Relacionamentos entre Classes (Model)

- Usuario (1) --- (1) Funcionario (funcionário pode ser um usuário)
 - Animal (1) --- (N) RegistroPeso (um animal tem muitos registros de peso)
 - Animal (1) --- (N) AplicacaoProduto (um animal recebe várias aplicações)
 - Animal (1) --- (N) AnimalDoente (um animal pode ficar doente várias vezes)
 - VacinaMedicamento (1) --- (N) AplicacaoProduto
 - Veterinario (herda de Usuario) --- (N) VisitaVeterinaria
 - VisitaVeterinaria (1) --- (N) AnimalDoente (uma visita pode diagnosticar vários animais)
 - ProducaoDiaria pode referenciar um Lote (não explicitado, mas implícito)
 - Estoque --- Alimento (relacionamento de produto armazenado)
 - Relatorio é gerado a partir de dados das demais classes.
-

4. Regras de Negócio Implementadas nos Métodos

- EstoqueController.registrarSaida() → verifica se quantidadeSaida <= quantidadeAtual. Caso contrário, lança exceção "Estoque não pode ficar negativo".
 - AnimalController.excluirAnimal() → verifica se o usuário logado é ADMIN antes de permitir exclusão.
 - ProducaoController.registrarProducao() → valida que o campo data é obrigatório (não pode ser nulo).
 - AnimalController.cadastrarAnimal() → gera identificacaoUnica automática (ex: GRANJA-001) e verifica unicidade.
 - AuthController.verificarPermissao() → apenas administradores podem acessar FuncionarioController e métodos de exclusão.
-

5. Observações sobre a Implementação MVC

- A camada Model deve conter apenas lógica de negócio e acesso a dados (DAO/Repository). Nenhuma interface gráfica.
 - A camada View captura entradas do usuário e as repassa ao Controller apropriado. Não deve conter lógica de negócio.
 - A camada Controller recebe requisições da View, chama métodos do Model e retorna os resultados para a View.
 - A persistência pode ser feita com banco de dados relacional (ex: MySQL, PostgreSQL) utilizando ORM (Hibernate, Entity Framework) ou JDBC.
-

Elaborado por: Rhauany Aragão, Jamilly Santos, Maria Rafaela, Mariano Santos

Disciplina: Análise e Projeto de Sistemas

Data: 03/06/2027